

KONFIGURATION DER SOFTWAREENTWICKLUNG UND QUALITÄTSPLANUNG

Meilenstein 3

ISEF01: Projekt Software Engineering

Thema E

Geschenke-Manager

Eine datenschutzorientierte, browserbasierte Webanwendung

Julian Ammann	31911823
Yin Yin Wu-Hanke	42303256
Anton Hirsch	IU14119299
Kevin Jordan Taghu	IU14150888

Tutor: Prof. Dr. rer. nat. Tobias Brückmann

Datum: 28. September 2026

Inhaltsverzeichnis

1 Konfiguration der Softwareentwicklung	1
1.1 Ausgangslage und Ziel	1
1.2 Vorgehensmodell	1
1.3 Anforderungsmanagement	1
1.4 Rollen und Zuständigkeiten in der Entwicklung	2
1.5 Architektur	3
1.6 Technologie-Stack	4
1.7 Repository-Struktur	5
1.8 Konfiguration des Backends	5
1.8.1 Schichten	5
1.8.2 Verbindliche Regeln	5
1.8.3 API-Konventionen	6
1.8.4 Authentifizierung und Sitzungen	6
1.8.5 Datenmodell und Migrationen	7
1.8.6 Scheduler, E-Mail und Bilder	8
1.9 Konfiguration des Frontends	8
1.9.1 Datenfluss	8
1.9.2 Verbindliche Regeln	9
1.9.3 Routen und Sitzung	9
1.10 Entwicklungsumgebung	10
1.11 Versionsverwaltung, Branching und Review	10
1.12 Lokale Qualitätssicherung über Git-Hooks	11
1.13 Continuous Integration	11
1.14 Abhängigkeiten	11
1.15 Konfiguration und Geheimnisse	12
1.16 Teststrategie	13
1.17 Bereitstellung und Betrieb	14
1.17.1 Container	14
1.17.2 Compose-Konfiguration	14
1.17.3 Hosting	14
1.18 Definition of Done	15

2 Konfiguration der Liefergegenstände	16
2.1 Liefergegenstände und Zuständigkeiten	16
2.2 Gliederung, Inhalte und Umfang	17
2.3 Dokumentation aus dem Code	18
2.4 Identifikation und Versionierung	19
2.5 Baselines je Meilenstein	19
2.6 Änderungs- und Freigabeprozess	19
2.7 Prüfverfahren für Liefergegenstände	20
2.8 Schutz vertraulicher Liefergegenstände	20
3 Qualitätsplanung	21
3.1 Qualitätsziele	21
3.2 Qualität des Softwaresystems	23
3.3 Qualität der Liefergegenstände	24

1 Konfiguration der Softwareentwicklung

1.1 Ausgangslage und Ziel

Dieses Kapitel legt fest, wie das Team den Geschenke-Manager bis MS 4 entwickelt, prüft und bereitstellt. Es beschreibt den geplanten Zielzustand. Alle Festlegungen gelten ab MS 3 verbindlich für alle Teammitglieder.

Zu MS 3 besteht ein technisches Grundgerüst: ein gemeinsames Repository für Backend und Frontend, gepinnte Werkzeugversionen, Git-Hooks, CI-Workflows, Dockerfiles und Health-Endpunkte. Die übrigen technischen und fachlichen Funktionen sind geplant und entstehen bis MS 4.

1.2 Vorgehensmodell

Das Team arbeitet iterativ in vier Abschnitten. Jeder Abschnitt endet mit einem lauffähigen Stand auf `main`.

Abschnitt	Zeitraum	Ziel
Sprint 0	bis 27.09.2026 (MS 3)	Entwicklungskonfiguration, verbindliches Datenmodell, Authentifizierung, leere Anwendung (Walking Skeleton) unter HTTPS erreichbar
Sprint 1	28.09. bis 03.10.2026	Mandantentrennung als wiederverwendbares Muster, erste Fachdomäne „Personen und Anlässe“ durch alle Schichten bis ins Frontend
Sprint 2	04.10. bis 08.10.2026	Geschenke, Beschenkungen, Aufgaben, Notizen, Anhänge, Benachrichtigungen, Teilen-Links, HTML-Ansicht und Geschenkvorschlüsse parallel
Stabilisierung	09.10. bis 11.10.2026	Feature-Freeze ab 08.10.2026. Nur noch Fehlerbehebung, Tests, Dokumentation und Bereitstellung für MS 4

Die erste Fachdomäne dient als Vorlage für alle weiteren. Erst wenn sie mit Tests und Frontend-Anbindung fertig ist, beginnen die übrigen Domänen parallel, und alle vier Entwickler schreiben ihre Modelle und Migrationen nach diesem erprobten Muster.

Jeder Abschnitt beginnt mit einem Regelmeeting in Microsoft Teams. Dort verteilt das Team die Aufgaben und prüft den Fortschritt gegen den Projektstrukturplan. Kurzfristige Blocker klärt das Team über Signal. Abnahmekriterien jedes Abschnitts sind die Qualitätsziele QZ-01 bis QZ-09 und die Definition of Done am Ende dieses Kapitels.

1.3 Anforderungsmanagement

Grundlage sind die Anforderungen aus den in MS 1 festgelegten funktionalen Anforderungen F-01 bis F-19 und Qualitätsanforderungen Q-01 bis Q-09.

Priorisierung: Die Priorisierung aus MS 1 gilt weiterhin. MUSS kennzeichnet dabei den verbindlichen Kernumfang (F-01 bis F-17). SOLL kennzeichnet Erweiterungen (F-18, F-19), die nur umgesetzt werden, wenn der Kernumfang und die Qualität dadurch nicht gefährdet sind.

Konkretisierung: Die Anforderungen aus MS 1 sind bewusst auf grober Ebene formuliert. Das Team konkretisiert sie schrittweise während der Umsetzung, jeweils zu Beginn des Abschnitts, in dem sie umgesetzt werden. Dabei werden sie in umsetzbare Aufgaben zerlegt und um prüfbare Akzeptanzkriterien ergänzt. Aus den Akzeptanzkriterien entstehen die Testfälle. Offene fachliche Fragen klärt das Team in der Planungsphase. Fragen, die nur der Tutor beantworten kann, stellt die Projektleitung über Redmine.

Nachvollziehbarkeit: Die Kennungen aus MS 1 bleiben über das gesamte Projekt hinweg erhalten. Aufgaben, Änderungen und Tests verweisen auf die betroffenen Anforderungen. Aufgaben werden als GitHub Issues im Projekt-Repository geführt und im GitHub Project „Geschenke-Manager Backlog“ geplant. Jedes Issue nennt im Titel oder im Feld „Anforderung“ die Anforderungs-ID und enthält die Akzeptanzkriterien als Checkliste; die PSP-Nummer steht in der Beschreibung. Das Project führt je Issue die Felder Abschnitt, Zieltermin, Priorität (MUSS oder SOLL), Bereich und Status. Die verantwortliche Person wird dem Issue zugewiesen. Das Product Backlog umfasst alle offenen Issues des Projects; das Sprint Backlog sind die Issues, deren Feld „Abschnitt“ auf den laufenden Abschnitt gesetzt ist. Die Bugliste sind die Issues mit dem Label `bug`; sie enthalten Reproduktionsschritte sowie Verweise auf Anforderung und Pull Request. Blocker werden ebenfalls als Issue erfasst und zusätzlich über Signal gemeldet. Pull Requests schließen ihre Issues über `Closes #<Nummer>`. Testfälle liegen in `backend/tests/` beziehungsweise den Frontend-Tests und führen die geprüften Anforderungs-IDs. Die Rückverfolgbarkeitsmatrix aus dem Testlauf (QZ-01) zeigt, ob eine Anforderung erfüllt ist. Testprotokolle werden aus den CI-Läufen übernommen und im Testabschlussbericht dokumentiert. Wesentliche Projektentscheidungen werden als GitHub Issue mit Begründung festgehalten, die Prüfungen der Liefergegenstände samt Ergebnis im Pull Request oder am zugehörigen Redmine-Ticket. Damit wird Q-09 abgedeckt.

Qualitätssicherung: Bevor eine Anforderung umgesetzt wird, wird die Konkretisierung darauf geprüft, ob sie eindeutig, testbar und widerspruchsfrei ist und ob Akzeptanzkriterien vorliegen. Nach der Umsetzung wird die fachliche Korrektheit gegen die Anforderung geprüft.

Änderungen: Neue oder geänderte Anforderungen bewertet das Team in der nächsten Planung und dokumentiert die Entscheidung nachvollziehbar. Reicht die Kapazität nicht aus, gilt die Reihenfolge aus dem Risikomanagement in MS 1 (PM-02, PM-03, PM-07):

1. SOLL-Anforderungen werden zurückgestellt.
2. Vom Team gewählte Ergänzungen werden vereinfacht oder als SOLL eingeordnet.
3. Die Anforderungen des Themenblatts bleiben verbindlich.

Ab dem Feature-Freeze am 08.10.2026 werden keine neuen Anforderungen mehr aufgenommen.

1.4 Rollen und Zuständigkeiten in der Entwicklung

Teammitglied	Entwicklung	Review und Prüfung
Julian Ammann	Einrichtung von Repository, Werkzeugen, CI/CD, Containern und Hosting (PSP 4.1). Authentifizierung und Sitzungen (PSP 4.2). Frontend	Reviewt Backend-Pull-Requests sowie Frontend-Pull-Requests, die er nicht selbst erstellt hat. Pflicht-Reviewer für Container-, Workflow- und Deploy-Konfiguration. Führt die Deploys aus
Kevin Jordan Taghu	Backend: Personen und Anlässe (PSP 4.3), Benachrichtigungen mit Scheduler und E-Mail-Versand (PSP 4.5), Teilen-Links und HTML-Ansicht (PSP 4.6)	Reviewt die Backend-Pull-Requests von Anton Hirsch. Koordiniert als Projektleitung die Abschnitte
Anton Hirsch	Backend: Geschenke, Beschenkungen, Aufgaben, Notizen und Anhänge (PSP 4.4), Geschenkvorschlge (PSP 4.7). Fachliche Testflle	Reviewt die Backend-Pull-Requests von Kevin Jordan Taghu. Prft die fachliche Korrektheit gegen die Anforderungen
Yin Yin Wu-Hanke	Frontend, Berechtigungs- und Sicherheitstests, Auswertung der Secret- und Abhngigkeitsscans	Reviewt Frontend-Pull-Requests, die sie nicht selbst erstellt hat. Prft sicherheitsrelevante nderungen (Anmeldung, Teilen-Links, Uploads)

Jeder Pull Request braucht die Freigabe einer anderen Person. Backend-Code prft Julian Ammann oder der jeweils andere Backend-Entwickler. Frontend-Code prft Julian Ammann oder Yin Yin Wu-Hanke, jeweils nicht die erstellende Person. Sicherheitsrelevante nderungen an Anmeldung, Teilen-Links und Uploads prft Yin Yin Wu-Hanke zustzlich aus Sicherheitssicht.

Die Backend-Strnge sind nach Fachdomnen getrennt. Jede Domne erhlt in jeder Schicht eine eigene Datei, etwa fr Personen je ein Modul fr Modell, Schema, Service und Router. Gemeinsam bearbeitet werden nur die beiden Dateien, in denen Modelle und Router registriert werden. Nur dort knnen Merge-Konflikte entstehen.

Backend und Frontend verbindet die OpenAPI-Spezifikation, die das Backend aus dem Code erzeugt. Ein Endpunkt gilt als bergeben, sobald er mit Zusammenfassung, Beschreibung und Beispielen in der Spezifikation steht. Das Frontend verwendet ausschlielich die daraus generierten Typen.

1.5 Architektur

```

Browser --HTTPS--> Traefik --> Frontend (SvelteKit, Node, :3000)
      |
      | serverseitige Datenabfragen und Formulare
      | Proxy fr Browser-Aufrufe auf /api/*
      v
Backend (FastAPI, :8000) --> PostgreSQL 17
      ^                   ^
      |                   |
Scheduler (gleiches Image) ----+
      |

```

- Öffentlich erreichbar ist nur das Frontend. Backend, Scheduler und Datenbank liegen in einem internen Container-Netz.
- Der Browser sieht nur eine Adresse. Das Session-Cookie ist deshalb ein First-Party-Cookie, und eine CORS-Freigabe entfällt.
- Der Scheduler läuft als eigener Prozess aus dem Backend-Image, immer mit genau einer Instanz. Im API-Prozess läuft kein Scheduler.
- Bilddateien liegen auf einem eigenen Volume und werden über eine geschützte API-Route ausgeliefert: mit gültiger Sitzung oder, soweit ausdrücklich freigegeben, mit gültigem Freigabe-Link.

1.6 Technologie-Stack

Bereich	Festlegung	Version
Backend-Sprache	Python	3.14
Web-Framework	FastAPI mit Uvicorn, vollständig asynchron	≥ 0.141
Datenzugriff	SQLAlchemy 2.0 (async) mit asyncpg	2.x
Datenbank	PostgreSQL	17
Schemamigrationen	Alembic	≥ 1.20
Validierung und Konfiguration	Pydantic, pydantic-settings	2.x
Passwort-Hashing	Argon2id	-
Logging	structlog, lesbar in der Entwicklung, JSON in Produktion	≥ 26.1
Scheduler	APScheduler in eigenem Prozess	-
E-Mail	aiosmtp, lokal Mailpit als Test-Postfach	-
Bildverarbeitung	Pillow für Formatprüfung und Entfernung von EXIF-Daten	-
Frontend-Sprache	TypeScript	6.x
Frontend-Framework	SvelteKit 2 mit Svelte 5, serverseitiges Rendering auf Node	2.x / 5.x
Styling	Tailwind CSS	4.x
API-Client	openapi-fetch mit aus der Spezifikation generierten Typen	0.17 / 7.x
Laufzeit Frontend	Node.js	26
Paketverwaltung	uv (Backend), pnpm (Frontend)	0.12.9 / 12.5.1
Tests	pytest, httpx, testcontainers, Vitest, Playwright	-

Python 3.14, Node 26 und TypeScript 6 sind sehr neue Versionen. Lockfiles und gepinnte Werkzeuge begrenzen das Risiko. Tritt ein blockierender Fehler auf, wechselt das Team für das betroffene Werkzeug auf die vorige Major-Version.

1.7 Repository-Struktur

```

isefgm/
+-- backend
|   +-- alembic/versions      Migrationen
|   +-- src/giftmanager
|   |   +-- api/v1           Router und Endpunkte
|   |   +-- core             Konfiguration, Datenbank, Anmeldung, Logging
|   |   +-- models           Datenbankmodelle
|   |   +-- schemas          Request- und Response-Schemas
|   |   +-- services         Geschäftslogik
|   +-- tests
+-- frontend
|   +-- src/lib/api           API-Client und generierte Typen
|   +-- src/routes           Seiten und Proxy für /api/*
+-- deploy                   Produktionskonfiguration und Betriebsanleitung
+-- docs                     Meilenstein-Dokumente, Datenmodell, generierte Dokumentation

```

1.8 Konfiguration des Backends

1.8.1 Schichten

Schicht	Verantwortung	Darf nicht
API	HTTP-Routing, Anmeldeprüfung, Request- und Response-Schemas, Statuscodes	Geschäftslogik enthalten oder SQL formulieren
Schemas	Validierung und Serialisierung, mit Beschreibungen und Beispielen	auf die Datenbank zugreifen
Services	Geschäftsregeln, Besitzerprüfung, Datenbank-abfragen	HTTP-Fehler werfen, Transaktionen abschließen, die Uhrzeit abfragen
Modelle	Tabellen, Beziehungen, Constraints, Kommentare	Logik enthalten

1.8.2 Verbindliche Regeln

Diese Regeln prüft das Team in jedem Backend-Review.

1. **Transaktion.** Jeder Request läuft in genau einer Datenbanktransaktion. Endet der Endpunkt normal, wird sie bestätigt, bei einem Fehler zurückgerollt. Services schließen keine Transaktionen selbst ab. Der Scheduler verwendet eine eigene Transaktion je Nachricht.
2. **Mandantentrennung (QZ-06).**

- Jede Tabelle mit Nutzerdaten erhält eine Spalte `owner_id` mit Fremdschlüssel auf das Konto. Wird das Konto gelöscht, werden die Daten mitgelöscht.
 - Services erhalten `owner_id` als ersten Parameter und filtern jede Abfrage danach. Ein Zugriff nur über die ID ohne Besitzerfilter ist verboten.
 - Fremde IDs beantwortet die API mit 404, nie mit 403, weil ein 403 verraten würde, dass der Datensatz existiert.
 - Beim Verknüpfen zweier Datensätze prüft der Service, dass beide demselben Konto gehören. Ausgenommen sind die systemweiten Anlassstypen.
 - Zu jeder Ressource gehört ein Test „Nutzer B greift auf Daten von Nutzer A zu → 404“.
3. **Fehler.** Fachliche Fehler haben eine gemeinsame Basisklasse. Ein zentraler Handler übersetzt sie, Validierungsfehler und HTTP-Fehler in ein einheitliches Format nach RFC 9457 (Problem Details) mit Typ, Titel, Status, Beschreibung und optional einer Liste von Feldfehlern.
 4. **Listen (QZ-02).** Jede Liste ist paginiert, standardmäßig 50 und höchstens 200 Einträge pro Seite. Die Antwort enthält die Einträge, die Gesamtzahl und die Seitenparameter. Beziehungen werden gebündelt geladen, nicht einzeln in Schleifen.
 5. **Zeit.** Zeitpunkte werden in UTC gespeichert, Kalenderdaten als reines Datum. Die aktuelle Zeit liefert eine austauschbare Uhr, in Tests etwa ein festes Datum für Weihnachtsbenachrichtigungen. Kalenderauswertungen laufen in der Zeitzone Europe/Berlin.
 6. **Dokumentation im Code.** Jede Tabelle und Spalte erhält einen Datenbankkommentar. Jeder Endpunkt erhält Zusammenfassung und Beschreibung, jedes Schema-Feld Beschreibung und Beispiel. Jeder Test nennt die Anforderungs-IDs, die er prüft. Diese Texte sind Deutsch, Code und Code-Kommentare Englisch.

1.8.3 API-Konventionen

- Alle Endpunkte liegen unter `/api/v1`. Ressourcen heißen im Plural, zum Beispiel `/persons`, `/gifts`, `/giftings`, `/tasks`.
- Jeder Endpunkt hat eine eindeutige Operation-ID in camelCase, etwa `listPersons` oder `createGift`. Daraus entstehen die Methodennamen im Frontend-Client.
- Schreibende Endpunkte geben die vollständige Ressource zurück. Nur Löschungen antworten mit 204 ohne Inhalt.
- Statuswerte sind englische Schlüssel wie `idea`, `planned`, `acquired`, `given`. Die deutsche Anzeige übernimmt das Frontend.
- Kalenderdaten haben das Format `YYYY-MM-DD`, Zeitpunkte ISO 8601 mit Zeitzone.

1.8.4 Authentifizierung und Sitzungen

Alle kontogebundenen fachlichen Endpunkte setzen eine Anmeldung voraus. Ausnahmen sind Registrierung, Anmeldung und die Nur-Lese-Ansicht über einen gültigen Freigabe-Link. Diese Ansichten

geben ausschließlich die ausdrücklich freigegebenen Personen und Geschenkideen zurück; Schreibzugriffe bleiben ausgeschlossen. Julian Ammann setzt die Authentifizierung in Sprint 0 um.

Aspekt	Festlegung
Passwörter	gehasht mit Argon2id, Länge 8 bis 128 Zeichen
Anmeldename	E-Mail-Adresse, in Kleinbuchstaben gespeichert, eindeutig
Sitzung	eigene Tabelle mit Erstellungs-, Ablauf- und letztem Nutzungszeitpunkt. Die Datenbank speichert nur einen Hash des Sitzungstokens
Cookie	HttpOnly, SameSite=Lax, außerhalb der Entwicklung Secure. Laufzeit 14 Tage, Verlängerung bei Nutzung
Schutz vor Ausspähen	Auch bei unbekannter E-Mail wird ein Passwort-Hash geprüft. Die Antwortzeit verrät nicht, ob ein Konto existiert
Rate-Limit	höchstens 10 Anmelde- oder Registrierungsversuche je E-Mail-Adresse in 15 Minuten
CSRF-Schutz	Zusätzlich zu SameSite prüft das Backend bei schreibenden Aufrufen, ob die Anfrage von der eigenen Adresse stammt
Endpunkte	Registrieren, Anmelden, Abmelden, eigenes Konto abrufen, Konto löschen (F-17)
Kontolöschung	löscht das Konto samt Sitzungen, allen zugehörigen Daten und hochgeladenen Bildern

Automatisierte Tests decken mindestens ab: Registrierung, doppelte E-Mail, zu kurzes Passwort, falsches Passwort, unbekanntes Konto, Anmeldung, Abmeldung, abgelaufene Sitzung, Rate-Limit, Kontolöschung und Zugriff ohne Anmeldung.

1.8.5 Datenmodell und Migrationen

Das Team legt in Sprint 0 ein vollständig attribuiertes Datenmodell als ER-Diagramm fest. Es baut auf dem Zielmodell aus MS 1 auf und ergänzt die zentrale Entität Beschenkung (*gifting*), die Zuordnungstabellen, Notizen, das Nutzerprofil und eine Kategorie für Geschenke als Grundlage der Vorschläge (F-16). Ab MS 3 ist dieses Modell für alle Entwickler verbindlich.

Konventionen:

- Primärschlüssel `id` als UUIDv7.
- Die Kontotabelle heißt `user_account`, weil `user` in PostgreSQL reserviert ist.
- Statuswerte werden als Text gespeichert und per `CHECK`-Constraint auf erlaubte Werte begrenzt: Beschenkung `idea` | `planned` | `acquired` | `given`, Aufgabe `open` | `in_progress` | `done` | `discarded`, Benachrichtigung `planned` | `sending` | `sent` | `failed`.
- Constraint-Namen folgen einer festen Konvention (`pk_`, `fk_`, `uq_`, `ix_`, `ck_`), damit Migrationen reproduzierbar bleiben.
- Wiederkehrende Anlässe speichern die Regel, die Beschenkung speichert den konkreten Termin.

- Die systemweiten Anlasstypen „Geburtstag“ und „Weihnachten“ legt eine Migration an.

Ablauf einer Schemaänderung:

1. Modell in der Datei der eigenen Domäne anlegen und registrieren.
2. Migration automatisch aus dem Modell erzeugen.
3. Migration von Hand gegenlesen, weil die automatische Erzeugung keine Umbenennungen und keine Datenänderungen erkennt.
4. Migration einspielen und prüfen, dass Modell und Datenbank übereinstimmen.
5. Im Review gleicht Julian Ammann Modell und Migration mit dem ER-Diagramm ab.

In Produktion spielt ein eigener Container die Migrationen ein, bevor die API startet.

1.8.6 Scheduler, E-Mail und Bilder

Diese Teile entstehen in Sprint 2.

Der Scheduler (PSP 4.5, Kevin Jordan Taghu) arbeitet in zwei Schritten. Zuerst legt er fällige Benachrichtigungen als Zeilen an, danach versendet er sie. Ein eindeutiger Schlüssel aus Konto, Bezug, Termin, Typ und Intervall verhindert doppelte Zeilen. Beim Versand sperrt jede Instanz nur die Zeilen, die sie gerade bearbeitet. Auch wenn versehentlich zwei Scheduler laufen, geht keine Nachricht doppelt hinaus (QZ-05). Bleibt eine Nachricht länger als 15 Minuten im Versand hängen, protokolliert der Scheduler eine Warnung, statt sie erneut zu senden.

Der E-Mail-Versand (PSP 4.5, Kevin Jordan Taghu) liegt hinter einer Schnittstelle mit drei Umsetzungen: SMTP für Produktion, Mailpit als lokales Test-Postfach für Entwicklung und End-to-End-Tests und ein Aufzeichner für Unit-Tests. Tests und Demos schreiben nie echte Adressen an. Für Produktion stehen Brevo, Resend und Mailgun zur Wahl, als Rückfallebene der SMTP-Zugang eines Teammitglieds. Die Wahl fällt zu Beginn von Sprint 2.

Für Bilder (PSP 4.4, Anton Hirsch) sind JPEG, PNG und WebP bis 5 MB erlaubt. Das Format prüft das Backend am Dateiinhalt, nicht an der Endung. Jedes Bild wird neu kodiert, dabei fallen EXIF-Daten wie Aufnahmeort weg. Dateien liegen je Konto getrennt unter einem zufälligen Namen und sind nur mit gültiger Sitzung oder gültigem Teilen-Link abrufbar.

1.9 Konfiguration des Frontends

Das Frontend setzen Julian Ammann und Yin Yin Wu-Hanke gemeinsam um und prüfen sich gegenseitig.

1.9.1 Datenfluss

```
Browser --> SvelteKit (Node, :3000) --> FastAPI (:8000)
|
```

```

+- Seitenaufbau und Formulare: serverseitiger Aufruf des Backends
|
+- Aufrufe des Browsers auf /api/*: Weiterleitung an das Backend

```

1.9.2 Verbindliche Regeln

1. Seiten laden ihre Daten serverseitig. Der API-Client wird pro Anfrage erzeugt und nicht global geteilt, weil sich sonst beim serverseitigen Rendern die Sitzungen verschiedener Nutzer vermischen könnten.
2. Schreibende Aktionen sind HTML-Formulare, die SvelteKit serverseitig verarbeitet. JavaScript verbessert sie nur, nötig ist es nicht, und alles lässt sich per Tastatur bedienen (QZ-04).
3. Fehlermeldungen des Backends zeigt das Frontend in verständlichem Deutsch am betroffenen Formular. Screenreader kündigen sie an. Eine zentrale Fehlerseite fängt unerwartete Fehler ab.
4. Typen stammen ausschließlich aus der generierten Typdatei. Sie wird nie von Hand bearbeitet.
5. Statuswerte der API übersetzt das Frontend ins Deutsche.

1.9.3 Routen und Sitzung

Bereich	Zweck	Schutz
Anmeldung, Registrierung	Konto anlegen und anmelden	öffentlich. Angemeldete Nutzer werden zur Übersicht weitergeleitet
Übersicht, fachliche Seiten	alle kontogebundenen Funktionen des Geschenke-Managers	nur mit Sitzung, sonst Weiterleitung zur Anmeldung mit Rücksprungziel
Geteilte Ansicht	ausdrücklich freigegebene Personen und Geschenkkideen anzeigen	öffentlich nur mit gültigem, nicht er-ratbarem Freigabe-Link; Nur-Lese-Zugriff
Konto	Konto anzeigen und löschen (F-17)	nur mit Sitzung
Abmeldung	Sitzung beenden, Cookie löschen	nur als Formularaktion
/api/*	Weiterleitung von Browser-Aufrufen an das Backend	Cookie wird durchgereicht

Bei jeder Anfrage prüft das Frontend serverseitig das Session-Cookie beim Backend und stellt den angemeldeten Nutzer allen Seiten bereit. Nach der Anmeldung leitet es nur auf eigene Pfade weiter, damit kein Link auf fremde Seiten umlenken kann (Open Redirect). Außerdem setzt es Sicherheits-Header gegen Einbettung in fremde Seiten und gegen falsch erkannte Dateitypen.

Die fachlichen Seiten entstehen in derselben Reihenfolge wie die Backend-Domänen. Das Frontend beginnt mit einer Domäne, sobald deren Endpunkte in der OpenAPI-Spezifikation stehen.

1.10 Entwicklungsumgebung

Jedes Teammitglied nutzt die eigene IDE. Damit alle mit denselben Werkzeugen arbeiten, pinnt das Werkzeug mise alle Versionen zentral im Repository:

Werkzeug	Version
Python	3.14
Node.js	26.9.0
pnpm	12.5.1
uv	0.12.9
Lefthook	2.1.14
gitleaks	8.30.1

Zusätzlich braucht jedes Teammitglied Docker mit Compose. Die Einrichtung eines neuen Rechners umfasst vier Schritte: Repository klonen, Werkzeuge über mise installieren, Abhängigkeiten und Git-Hooks mit einem gemeinsamen Setup-Befehl einrichten, lokale Datenbank als Container starten. Danach laufen Backend und Frontend mit automatischem Neuladen lokal.

Für wiederkehrende Aufgaben gibt es gemeinsame Befehle:

Befehl	Zweck
<code>mise run check</code>	Lint, Typprüfung und Tests für Backend und Frontend
<code>mise run openapi</code>	OpenAPI-Spezifikation exportieren und Frontend-Typen erzeugen
<code>mise run docs</code>	Dokumentation aus dem Code erzeugen
<code>mise run docker</code>	Container-Images bauen

Eine lokale Compose-Ergänzung öffnet die Ports für Datenbank, Backend und Frontend und startet im Entwicklungsmodus. Die Produktion verwendet sie nicht.

1.11 Versionsverwaltung, Branching und Review

- Der Branch `main` ist geschützt. Änderungen kommen nur per Pull Request mit mindestens einer Freigabe hinein. Force-Pushes und das Löschen des Branches sind gesperrt.
- Für Container-, Workflow- und Deploy-Dateien ist Julian Ammann als Code Owner und Pflicht-Reviewer eingetragen, weil diese Dateien bestimmen, was auf dem Produktionsserver läuft.
- Branches sind kurzlebig und heißen `feature/<domäne>-<thema>`, `fix/<thema>`, `docs/<thema>` oder `chore/<thema>`.
- Commit-Nachrichten folgen Conventional Commits mit der Komponente als Scope, zum Beispiel `feat(backend): ..., fix(frontend): ..., docs(ms3): ....`
- Ein Pull Request umfasst eine Domäne oder ein Thema. Er wird zusammengeführt, wenn die CI grün ist, die Reviews laut Rollentabelle vorliegen und alle Anmerkungen bearbeitet sind.

1.12 Lokale Qualitätssicherung über Git-Hooks

Zeitpunkt	Prüfung	Komponente
vor Commit	Ruff Lint mit Auto-Fix und Ruff Format	Backend
vor Commit	ESLint mit Auto-Fix und Prettier	Frontend
vor Commit	Neuerzeugung der Frontend-Typen, wenn sich Endpunkte oder Schemas ändern	Backend → Frontend
vor Commit	Secret-Scan der geänderten Dateien mit gitleaks	gesamt
vor Push	Typprüfung mit Pyright im Strict-Modus	Backend
vor Push	Prüfung auf fehlende Migrationen	Backend
vor Push	Typprüfung mit svelte-check	Frontend

Hooks lassen sich lokal umgehen. Die CI wiederholt deshalb alle Prüfungen.

1.13 Continuous Integration

GitHub Actions führt die Prüfungen nur für die geänderte Komponente aus. Bei reinen Dokumentationsänderungen meldet ein Platzhalter-Workflow die Pflichtprüfungen als erfolgreich, damit der Branchschutz nicht blockiert.

Workflow	Auslöser	Prüfungen
Backend	Änderungen im Backend	Lint, Typprüfung, Migrationen gegen PostgreSQL 17 einspielen und auf Vollständigkeit prüfen, Tests und Abhängigkeitsscan
Frontend	Änderungen im Frontend	generierte Typen aktuell, Formatierung, Lint, Typprüfung, Unit- und Komponententests im Browser, Build und Abhängigkeitsscan
Docker	Änderungen an Backend oder Frontend	Build beider Images, Veröffentlichung in der GitHub Container Registry bei <code>main</code> und Versions-Tags
Secrets	jeder Push auf <code>main</code> und jeder Pull Request	Secret-Scan über die gesamte Historie

Die Frontend-CI prüft, dass die generierten Typen zur OpenAPI-Spezifikation passen. Eine automatische Prüfung der OpenAPI-Spezifikation gegen den Backend-Code ist im bestehenden Backend-Workflow noch nicht eingerichtet und wird vor MS 4 ergänzt. Der API-Vertrag ist damit erst nach dieser Ergänzung auf beiden Seiten abgesichert. Alle Workflows laufen mit Leserechten, nur der Docker-Workflow darf zusätzlich Images veröffentlichen. Die CI nutzt dieselben gepinnten Werkzeugversionen wie die lokale Umgebung.

1.14 Abhängigkeiten

- Alle Abhängigkeiten sind in Lockfiles fixiert. Die CI installiert nur aus den Lockfiles.

- Entwicklungswerkzeuge sind von den Laufzeitabhängigkeiten getrennt und gelangen nicht in die Produktions-Images.
- Dependabot prüft künftig wöchentlich Python- und npm-Pakete, GitHub Actions und Docker-Basis-Images und öffnet gebündelte Pull Requests. Die Konfiguration wird vor MS 4 auf `main` eingerichtet.
- `pip-audit` und `pnpm audit` laufen in den bestehenden Workflows derzeit mit `continue-on-error`; Funde blockieren einen Merge daher noch nicht. Der Backend-Scan prüft zudem bisher die eigene Werkzeugumgebung von `pip-audit` statt der gesperrten Projektabhängigkeiten. Vor MS 4 werden beide Scans korrigiert und die Fehlerfortsetzungen entfernt. Im Frontend blockieren dann Befunde der Schweregrade hoch und kritisch einen Merge (`pnpm audit -audit-level=high`); mittlere und niedrige Befunde werden in der Bugliste mit Priorität und Zieltermin erfasst. `pip-audit` weist keine Schweregrade aus, deshalb blockiert im Backend jeder Befund. Begründete Ausnahmen dokumentiert die verantwortliche Person in einem Issue mit Risiko und Behebungsziel und hinterlegt sie per `-ignore-vuln`; Yin Yin Wu-Hanke prüft und genehmigt sie (QZ-06). Der Docker-Workflow baut derzeit Images, scannt sie aber noch nicht; ein Image-Scan wird vor MS 4 ergänzt.

1.15 Konfiguration und Geheimnisse

Die Anwendung liest ihre Konfiguration ausschließlich aus Umgebungsvariablen. Jede Variable ist im Code typisiert und beschrieben. Daraus entsteht automatisch die Konfigurationstabelle der Betriebsdokumentation.

Variable	Standard	Bedeutung
APP_ENV	development	development aktiviert die API-Dokumentation und lesbare Logs, production JSON-Logs
LOG_LEVEL	INFO	DEBUG, INFO, WARNING, ERROR
DATABASE_URL	lokale Entwicklungs-DB	Verbindung zu PostgreSQL
DEFAULT_TIMEZONE	Europe/Berlin	Zeitzone für Kalenderauswertungen
UPLOADS_DIR	/data/uploads	Ablage der Bilddateien
SESSION_TTL_DAYS	14	Laufzeit einer Sitzung in Tagen
SMTP_*	keiner	Zugang zum E-Mail-Anbieter
API_URL	http://localhost:8000	Adresse des Backends aus Sicht des Frontends
ORIGIN	keiner	öffentliche Adresse des Frontends, in Produktion Pflicht

Regeln für Geheimnisse:

- `.env`-Dateien sind von der Versionierung ausgeschlossen. Die Vorlage `.env.example` enthält nur unkritische Entwicklungswerte.
- `gitleaks` prüft vor jedem Commit und in der CI über die gesamte Historie.

- Produktionsgeheimnisse wie Datenbank- und SMTP-Zugang liegen nicht im Projekt-Repository, sondern verschlüsselt im separaten Infrastruktur-Repository. Sie werden erst beim Deploy eingespielt.
- Testkonten und Zugangsdaten für den Tutor übergibt das Team ausschließlich über Redmine.

1.16 Teststrategie

Ebene	Werkzeug	Umfang	Verantwortlich
Unit Backend	pytest mit Test-Uhr und Test-Postfach	Services, Planung der Benachrichtigungen, Vorschlagslogik	Entwickler der Domäne
Integration Backend	pytest, httpx, testcontainers	Kontogebundene Endpunkte mit Normalfall, Validierung und Fremdzugriff → 404. Freigabe-Endpunkte zusätzlich mit gültigem, ungültigem, abgelaufenem und widerrufenem Link; nur Lesen. Echte PostgreSQL-Datenbank, jeder Test wird danach zurückgerollt	Entwickler der Domäne
Berechtigung	wie Integration	Mandantentrennung je Ressource, Teilen-Link zeigt nur freigegebene Personen	Yin Yin Wu-Hanke
Unit Frontend	Vitest im Browser	Hilfsfunktionen und Komponenten	Julian Ammann
End-to-End	Playwright gegen den kompletten Stack	Kernabläufe: registrieren, Person anlegen, Idee → Beschenkung → verschenkt, Teilen-Link ohne Anmeldung öffnen, unzulässige Änderungen über den Link abweisen und Konto löschen. Chromium, Firefox und WebKit; Safari wird zusätzlich manuell geprüft (QZ-08)	Julian Ammann, Anton Hirsch
Last	Skript mit synthetischem Testbestand	Nachweis QZ-02, Messwerte gehen in den Testabschlussbericht	Anton Hirsch

- Ziel aus QZ-07 sind mindestens 70 % Zeilenabdeckung im Backend. Der bestehende Backend-Workflow misst diese Abdeckung noch nicht; `pytest-cov` und der Grenzwert werden vor MS 4 in der CI eingerichtet.
- Jeder Test nennt die Anforderungen, die er prüft. Sobald die Testkennungen in den Tests und im CI-Export verfügbar sind, entsteht aus jedem vollständigen Testlauf die Rückverfolgbarkeitsmatrix Anforderung ↔ Test ↔ Ergebnis als Nachweis für QZ-01.
- Warnungen gelten in den Backend-Tests als Fehler.
- End-to-End-Tests laufen nur auf `main` und vor Releases, weil sie Pull Requests sonst zu stark verlangsamen. Playwright prüft dabei WebKit als Annäherung an Safari, nicht den markengebundenen Safari-Browser; deshalb erfolgt die zusätzliche Safari-Prüfung manuell (siehe Playwright-Dokumentation zu Browsern).

1.17 Bereitstellung und Betrieb

1.17.1 Container

Image	Basis	Merkmale
Backend	python:3.14-slim	mehrstufiger Build, kein Root-Benutzer, nur Laufzeitabhängigkeiten. Dient auch für Migration und Scheduler
Frontend	node:26-alpine	mehrstufiger Build, kein Root-Benutzer

Die Images werden in der GitHub Container Registry mit Commit-Kennung und bei Releases mit der Versionsnummer veröffentlicht.

1.17.2 Compose-Konfiguration

Die Container-Konfiguration besteht aus drei Dateien:

Datei	Inhalt
Basis	Dienste Datenbank, Migration, API, Scheduler und Frontend. Health-Checks, internes Netz, Volumes für Datenbank und Bilder. Keine offenen Ports
Lokal	Build aus dem Quellcode, offene Ports, Entwicklungsmodus
Produktion	Anbindung des Frontends an den Reverse Proxy mit der öffentlichen Domain, Speicher- und CPU-Limits für alle Dienste

Health-Checks steuern die Startreihenfolge. Die API startet erst nach erfolgreicher Migration, das Frontend erst, wenn die API bereit ist. Zwei Health-Endpunkte melden, ob der Prozess läuft und ob die Datenbank erreichbar ist.

1.17.3 Hosting

Das System läuft auf einem bestehenden, gehärteten Server eines Teammitglieds. Dort übernimmt Traefik Routing, TLS-Zertifikate über Let's Encrypt, HSTS, Sicherheits-Header und ein Rate-Limit am Eingang. Firewall, Monitoring und Log-Sammlung stellt ebenfalls der Server. Das Projekt liefert nur seine Container-Konfiguration.

Julian Ammann stößt das Deploy im separaten Infrastruktur-Repository an, sobald die Images gebaut sind. Deployt wird immer ein fester Commit mit dem dazu gebauten Image, nie der jeweils aktuelle Stand von `main`. Weil das Deploy dort angestoßen wird, liegen im öffentlichen Projekt-Repository keine Deploy-Zugangsdaten. Vor jedem Deploy lehnen automatische Prüfungen offene Ports, Einbindungen von Host-Verzeichnissen und fremde Netze ab.

Schon in Sprint 0 ist die leere Anwendung mit Health-Endpunkt unter HTTPS erreichbar, damit Probleme bei der Bereitstellung nicht erst kurz vor MS 4 auffallen.

Das System verarbeitet nur synthetische Testdaten, ein externes Backup ist deshalb nicht vorgesehen. Nach einem Datenverlust spielt ein Seed-Skript Testkonten und Testbestand neu ein. Das Skript entsteht vor der Bereitstellung für den Tutor.

1.18 Definition of Done

Eine Funktion ist fertig, wenn alle Punkte erfüllt sind:

Backend

- ☐ Tabelle entspricht dem verbindlichen Datenmodell, Migration erzeugt, gegengelesen und vollständig
- ☐ jede Tabelle und Spalte hat einen Datenbankkommentar
- ☐ Service filtert jede Abfrage nach dem Besitzer, fachliche Fehler nutzen die gemeinsame Fehlerklasse
- ☐ Kontogebundener Endpunkt erfordert Anmeldung; öffentliche Freigabe-Endpunkte erlauben ausschließlich den über einen gültigen Link freigegebenen Nur-Lese-Zugriff. Jeder Endpunkt hat Operation-ID, Zusammenfassung und Beschreibung. Schema-Felder haben Beschreibung und Beispiel
- ☐ Listen sind paginiert
- ☐ Tests für Normalfall, Validierung und Fremdzugriff → 404, jeweils mit Anforderungs-ID
- ☐ OpenAPI-Spezifikation und generierte Dokumentation sind aktualisiert und committet

Frontend

- ☐ Daten werden serverseitig geladen, Änderungen laufen über Formularaktionen
- ☐ Fehler sind sichtbar und werden von Screenreadern angekündigt. Die Seite ist ohne JavaScript bedienbar
- ☐ Typen stammen nur aus der generierten Typdatei. Lint, Typprüfung und Tests sind grün

Gemeinsam

- ☐ CI grün, Review laut Rollentabelle freigegeben
- ☐ betroffene Anforderung steht in der Rückverfolgbarkeitsmatrix als bestanden

2 Konfiguration der Liefergegenstände

2.1 Liefergegenstände und Zuständigkeiten

Jeder Liefergegenstand erhält eine Kennung, eine verantwortliche und eine prüfende Person sowie einen festen Ablageort. Die prüfende Person ist nie die verantwortliche.

ID	Liefergegenstand	MS	Format	Ablage	Verantwortlich	Prüfend
LG-01	Konfiguration der Softwareentwicklung und Qualitätsplanung	3	Markdown → PDF	docs/	Julian Ammann	Kevin Jordan Taghu
LG-02	Programmcode Backend	4	Git-Repository	backend/	Kevin Jordan Taghu, Anton Hirsch (Anmeldung: Julian Ammann)	Julian Ammann
LG-03	Programmcode Frontend	4	Git-Repository	frontend/	Julian Ammann, Yin Yin Wu-Hanke	gegenseitiges Review
LG-04	Lauffähiges System (Link)	4	Container-Images, URL	GitHub Container Registry, Produktionsserver	Julian Ammann	Anton Hirsch
LG-05	Benutzerhandbuch	4	Markdown → PDF mit Screenshots	docs/ms4/	Yin Yin Wu-Hanke	Anton Hirsch
LG-06	Fachliche Dokumentation: Prozesse, Konzepte, Geschäftsregeln	4	Markdown → PDF	docs/ms4/	Anton Hirsch	Kevin Jordan Taghu
LG-07	Technische Dokumentation: Architektur, Komponenten, Schnittstellen, Datenbank	4	Markdown, teilweise generiert	docs/ms4/, docs/generated/	Julian Ammann (Architektur, Frontend), Kevin Jordan Taghu (Backend)	Anton Hirsch
LG-08	Betriebsdokumentation: Installation, Konfiguration, Admin-Account	4	Markdown, teilweise generiert	deploy/, docs/generated/	Julian Ammann	Kevin Jordan Taghu
LG-09	Testabschlussbericht mit Testfällen und Testprotokollen	4	Markdown → PDF, CI-Protokolle	docs/ms4/, docs/generated/	Anton Hirsch	Yin Yin Wu-Hanke
LG-10	Liste der Testkonten und Zugangsdaten	4	PDF	nur Redmine, nie im Repository	Yin Yin Wu-Hanke	Julian Ammann

ID	Liefergegenstand	MS	Format	Ablage	Verantwortlich	Prüfend
LG-11	Ergebnispräsentation: Projektablauf mit Teilergebnissen, Liefergegenständen und Reflexion ihrer Erstellung; technischer Überblick, Testabschlussbericht, Demo und Lessons Learned	5	Video oder Link, höchstens 20 Minuten; Demo höchstens 7 Minuten	docs/ms5/	Kevin Jordan Taghu	Anton Hirsch
LG-12	Gemeinsamer Projektbericht	6	PDF	docs/ms6/	Kevin Jordan Taghu	alle

2.2 Gliederung, Inhalte und Umfang

ID	Gliederung und Inhalte	Umfang
LG-01	Konfiguration der Softwareentwicklung, Konfiguration der Liefergegenstände, Qualitätsplanung (Gliederung dieses Dokuments)	alle in der Aufgabenstellung zu MS 3 genannten Inhalte
LG-02	Backend nach der Repository-Struktur: Migrationen, Router, Kern, Modelle, Schemas, Services, Tests	alle MUSS-Anforderungen mit Tests; Definition of Done je Funktion erfüllt
LG-03	Frontend nach der Repository-Struktur: API-Client mit generierten Typen, Seiten, Proxy für /api/*, Tests	alle Oberflächen zu den MUSS-Anforderungen
LG-04	Öffentliche Adresse, Container-Images mit Versionsnummer, Health-Endpunkte	im Browser ohne lokale Installation prüfbar
LG-05	Einstieg und Anmeldung, Personen und Anlässe, Geschenkideen und Beschenkungen, Aufgaben und Notizen, Benachrichtigungen, Teilen-Links, Konto löschen, häufige Fragen. Jeder Ablauf mit Schrittfolge und Screenshot	ein Kapitel je Kernablauf aus den End-to-End-Tests
LG-06	Fachliche Prozesse mit Sequenzdiagrammen, fachliche Konzepte (Person, Anlass, Geschenk/Idee, Besenkung, Teilen-Link), Geschäftsregeln mit Verweis auf die Anforderungs-IDs, Glossar	jede MUSS-Anforderung mindestens einer Regel oder einem Prozess zugeordnet
LG-07	Architekturüberblick und Architekturentscheidungen, Komponenten von Backend, Frontend und Scheduler, Schnittstellen (OpenAPI-Dokumentation), Datenbank (ER-Diagramm, Tabellen- und Spaltenbeschreibungen)	Schnittstellen und Datenbank vollständig generiert, übrige Kapitel von Hand

ID	Gliederung und Inhalte	Umfang
LG-08	Voraussetzungen, Installation aus leerem Zustand, Konfiguration (Umgebungsvariablen), Admin-Account, Aktualisierung, Neuaufbau des Testbestands über das Seed-Skript	reicht einer nicht beteiligten Person für die Installation (QZ-08)
LG-09	Testgegenstand und Testumgebung, Teststrategie, Testfälle je Ebene, Rückverfolgbarkeitsmatrix, Testprotokolle, Messwerte zu QZ-01 bis QZ-09, offene Fehler aus der Bugliste, Bewertung	alle Qualitätsziele mit Messwert und Ergebnis
LG-10	Je Konto Zweck, Anmeldename und Passwort; Admin-Account gesondert gekennzeichnet	alle für die Prüfung durch den Tutor benötigten Konten
LG-11	Projektablauf mit Teilergebnissen, Liefergegenständen und Reflexion; technischer Überblick; Überblick Testabschlussbericht; Demo; Lessons Learned	höchstens 20 Minuten, Demo höchstens 7 Minuten
LG-12	Titelblatt mit Angaben aller Mitglieder und Seitenbereichen, Einleitung, Projektverlauf MS 0 bis MS 5 mit Ergebnissen, Reflexion und Fazit	je Person ein zusammenhängender, namentlich zugeordneter Textteil von 7 bis 10 Seiten. Jedes Mitglied reicht den Bericht gesondert in Turnitin ein

2.3 Dokumentation aus dem Code

MS 4 verlangt acht Liefergegenstände, davon fünf Dokumente. Ein großer Teil davon entsteht direkt beim Entwickeln, sonst müsste das Team alles in der letzten Woche schreiben. Nach den Regeln zur Dokumentation im Code tragen Endpunkte, Tabellen, Konfiguration und Tests ihre Beschreibung von Anfang an. Ein gemeinsamer Befehl erzeugt daraus die Dokumente.

Liefergegenstand	Quelle im Code	Entstehung	Anteil aus Code
Technische Doku: Schnittstellen	Beschreibungen an Endpunkten und Schema-Feldern	generiert, dazu OpenAPI-Spezifikation und interaktive API-Doku	fast vollständig
Technische Doku: Datenbank	Kommentare an Tabellen und Spalten, ER-Diagramm	generiert	weitgehend
Technische Doku: Architektur, Komponenten	Architekturentscheidungen aus diesem Dokument	von Hand	teilweise
Betriebsdoku: Konfiguration	Beschreibungen der Umgebungsvariablen	generiert	vollständig
Betriebsdoku: Installation, Admin-Account	Compose-Dateien, Seed-Skript	von Hand	teilweise
Testabschlussbericht	Anforderungs-IDs an Tests, Testläufe, Abdeckung	generiert aus dem Testlauf, ergänzt um Bewertung	weitgehend
Fachliche Doku: Geschäftsregeln	Beschreibungen der Services, Fehlermeldungen, Testnamen	von Hand	teilweise

Liefergegenstand	Quelle im Code	Entstehung	Anteil aus Code
Fachliche Doku: Prozesse, Konzepte	keine	von Hand, mit Sequenzdiagrammen und Glossar	kaum
Benutzerhandbuch	Screenshots aus den End-to-End-Tests	Text von Hand, Screenshots automatisch	kaum

Die generierten Dateien liegen unter `docs/generated/` und werden committet, damit man sie ohne Werkzeuge lesen und Änderungen im Pull Request sehen kann. Von Hand bearbeitet werden sie nie. Die Screenshots für das Benutzerhandbuch erzeugen die End-to-End-Tests bei jedem Lauf neu, sodass sie nach Änderungen an der Oberfläche aktuell bleiben.

2.4 Identifikation und Versionierung

- Dokumente liegen unter `docs/` und heißen `ms<Nummer>_<thema>.md`. Ab MS 4 erhält jeder Meilenstein einen Unterordner `docs/ms<Nummer>/`. Dateinamen sind durchgehend klein geschrieben.
- PDF-Abgabefassungen werden aus den Markdown- oder LaTeX-Quellen erzeugt und unter `docs/pdf/` abgelegt. Änderungen erfolgen immer in der Quelle, nie im PDF.
- Die Software folgt Semantic Versioning, aktuell 0.1.0. Die Abgabe zu MS 4 erhält die Version 1.0.0. Das Setzen der Version löst Build und Veröffentlichung der Images mit derselben Versionsnummer aus. Korrekturen nach Tutorfeedback erhöhen die letzte Stelle, etwa 1.0.1.
- Jeder Abgabestand erhält einen Git-Tag `ms<Nummer>-abgabe`. Über den Tag lässt sich der eingereichte Stand jederzeit wiederherstellen, auch wenn `main` weiterentwickelt wird.

2.5 Baselines je Meilenstein

Baseline	Inhalt	Kennzeichnung
MS 3	dieses Dokument, Datenmodell, Entwicklungsumgebung, CI, Grundgerüst mit Health-Endpunkten	Tag <code>ms3-abgabe</code>
MS 4	Programmcode, Container-Images, Dokumentation LG-05 bis LG-09, generierte Dokumente	Tags <code>v1.0.0</code> und <code>ms4-abgabe</code> , Images <code>1.0.0</code>
MS 5	Präsentation auf Basis der MS-4-Baseline	Tag <code>ms5-abgabe</code>
MS 6	Projektbericht	Tag <code>ms6-abgabe</code>

Nach einer Baseline ändert sich der abgegebene Stand nur über einen neuen Pull Request und eine neue Version.

2.6 Änderungs- und Freigabeprozess

1. Die verantwortliche Person erstellt oder ändert den Liefergegenstand in einem eigenen Branch.

2. Sie öffnet einen Pull Request gegen `main`. Die prüfende Person prüft Inhalt, Form und Vollständigkeit gegen die Qualitätsziele und die Definition of Done.
3. Die CI muss grün sein.
4. Nach der Freigabe wird der Pull Request zusammengeführt, die Abgabefassung erzeugt und der Stand getaggt.
5. Die Projektleitung lädt die Abgabe in Redmine hoch, setzt das Meilensteinticket auf **Feedback** und weist es dem Tutor zu.
6. Das Tutorfeedback steht am Redmine-Ticket. Notwendige Korrekturen durchlaufen erneut die Schritte 1 bis 5. Der Tutor nimmt mit dem Status **closed** ab.

Die letzten zwei Tage vor jeder Abgabe sind für Review, Korrektur und Bereitstellung reserviert.

2.7 Prüfverfahren für Liefergegenstände

Liefergegenstand	Prüfung
Programmcode	CI grün (Lint, Typen, Tests, Schwachstellen- und Secret-Scan), Review laut Rollentabelle, Definition of Done erfüllt
Lauffähiges System	Installation aus leerem Zustand, Health-Endpunkte, End-to-End-Abläufe gegen die Produktionsadresse, Anmeldung mit allen Testkonten
Generierte Dokumente	aus dem Release-Stand erzeugt, Änderungen im Pull Request geprüft
Handgeschriebene Dokumente	eine nicht beteiligte Person liest den Text gegen den Code und gibt ihn im Pull Request frei
Testabschlussbericht	Zahlen zu Testanzahl, Abdeckung und Antwortzeiten nur aus echten Läufen des Release-Stands, Matrix vollständig
Testkonten	Anmeldung mit jedem Konto vor der Übergabe, keine Zugangsdaten im Repository

2.8 Schutz vertraulicher Liefergegenstände

Das Repository ist öffentlich. Zugangsdaten, Testkonten und Produktionsgeheimnisse werden deshalb nie versioniert. Testkonten erhält der Tutor nur über Redmine (LG-10), Produktionsgeheimnisse liegen im separaten Infrastruktur-Repository. Der Prototyp verarbeitet ausschließlich synthetische Testdaten. gitLeaks fängt lokal und in der CI Geheimnisse ab, die versehentlich in einen Commit geraten.

3 Qualitätsplanung

3.1 Qualitätsziele

Die Qualitätsziele konkretisieren die Qualitätsanforderungen Q-01 bis Q-09 aus MS 1 und sind nach den Merkmalen der ISO/IEC 25010 gegliedert. Die Kennungen QZ ergänzen die ursprünglichen Kennungen Q; sie ersetzen diese nicht. Jedes Ziel ist messbar und einem Prüfverfahren, einem Zeitpunkt und einer verantwortlichen Person zugeordnet.

Anforderung aus MS 1	Konkretisierung in MS 3
Q-01 Mandantentrennung	QZ-06 Sicherheit
Q-02 Benachrichtigungszuverlässigkeit	QZ-05 Zuverlässigkeit
Q-03 Sicherheit	QZ-06 Sicherheit
Q-04 Datenschutz	QZ-06 Sicherheit
Q-05 Leistung	QZ-02 Leistungseffizienz
Q-06 Wartbarkeit	QZ-07 Wartbarkeit
Q-07 Barrierefreiheit	QZ-04 Benutzbarkeit und Barrierefreiheit
Q-08 Zeitkorrektheit	QZ-09 Zeitkorrektheit
Q-09 Nachvollziehbarkeit	QZ-01 Rückverfolgbarkeit der Anforderungen sowie Dokumentations- und Freigaberegeln für Entscheidungen und Liefergegenstände

QZ-03 (API-Vertrag und HTML-Konformität) und QZ-08 (Übertragbarkeit und Browserdarstellung) ergänzen die MS-1-Ziele.

ID	Merkmal	Qualitätsziel	Prüfverfahren	Prüfer
QZ-01	Funktionale Eignung	Alle MUSS Anforderungen sind umgesetzt und durch mindestens einen Testfall abgedeckt; 100 % der zugehörigen Testfälle sind bestanden. Berechnungen sind korrekt, z. B. enthält die Geburtstagsübersicht des Folgemonats genau die betroffenen Personen. Die Umwandlung einer Geschenkidee in ein Geschenk erfolgt in einem Schritt.	Rückverfolgbarkeitsmatrix Anforderung <=> Testfall; Unit-, Integrations- und Systemtests	Yin Yin Wu-Hanke
QZ-02	Leistungseffizienz	Liste mit 100 Personen, 1.000 Geschenkideen und 1.000 Beschenkten werden in der festgehaltenen lokalen Testumgebung innerhalb von 2 Sekunden angezeigt (Q-05).	Nach einem Aufwärmtestlauf 30 Messungen vom Aufruf bis zur Anzeige der Liste; der 95. Perzentilwert muss höchstens 2 Sekunden betragen. Testgerät und Laufbedingungen werden im Testprotokoll festgehalten.	Julian Ammann

ID	Merkmal	Qualitätsziel	Prüfverfahren	Prüfer
QZ-03	Kompatibilität	Der Frontend-Client entspricht der OpenAPI-Spezifikation des Backends (typisierter Client, keine Typfehler). Der HTML-Export ist valides HTML.	Typprüfung in CI, W3C-Validator	Anton Hirsch, Kevin Jordan Taghu
QZ-04	Benutzbarkeit und Barrierefreiheit	Eine Geschenkidee kann ohne Anleitung in höchstens 3 Interaktionen und unter 30 Sekunden erfasst werden. Löschvorgänge erfordern eine Bestätigung, ungültige Eingaben erzeugen verständliche Fehlermeldungen, wesentliche Abläufe funktionieren per Tastatur und Zustände werden nicht nur durch Farbe vermittelt. Lighthouse-Accessibility-Wert ≥ 90 .	Drei externe Testpersonen führen dieselbe Erfassungsaufgabe aus; das Ziel gilt als erreicht, wenn alle drei die Erfassung innerhalb der vorgegebenen Zeit und Interaktionen ohne Hilfe abschließen. Zeit und Interaktionen werden einzeln protokolliert. Zusätzlich Testfälle für ungültige Eingaben und Tastaturbedienung sowie Lighthouse-Audit.	Anton Hirsch, Yin Yin Wu-Hanke
QZ-05	Zuverlässigkeit	Benachrichtigungen werden zum vorgesehenen Termin erzeugt. Wiederholte oder parallele Scheduler-Läufe erzeugen dabei keine zusätzlichen Versandaufträge. Unklare Versandergebnisse werden nicht automatisch erneut versendet, sondern protokolliert. Ein Ausfall des E-Mail-Versands beeinträchtigt die übrigen Funktionen nicht. Container starten nach einem Neustart automatisch.	Scheduler-Test mit simulierter Systemzeit, Integrationstest mit simuliertem Ausfall, Neustart-Test	Yin Yin Wu-Hanke
QZ-06	Sicherheit	Nutzende greifen ausschließlich auf eigene Daten zu; die einzige Ausnahme sind ausdrücklich freigegebene Inhalte über einen gültigen Freigabe-Link. Passwörter und Tokens werden nur gehasht gespeichert. Teilen-Links sind nicht erratbar, optional befristet und jederzeit deaktivierbar. Keine Secrets im Repository. Es werden ausschließlich synthetische Daten verarbeitet, Vorschläge nutzen keine personenbezogenen Daten anderer Konten.	Autorisierungstests je Endpunkt, pnpm audit, pip-audit, Image-Scan (vor MS 4 einzurichten), Secret-Scan, Checkliste Datenschutz	Kevin Jordan Taghu, Anton Hirsch

ID	Merkmal	Qualitätsziel	Prüfverfahren	Prüfer
QZ-07	Wartbarkeit	Linter und Typprüfung laufen ohne Fehler. Jeder Merge auf main erfolgt nach Code-Review. Die Zeilenabdeckung der Backend-Unit-Tests $\geq 70\%$ und jeder API-Endpunkt besitzt mindestens einen automatisierten Test. Frontend und Backend sind getrennt bau- und testbar.	ESLint, svelte-check, Ruff, pytest-cov, Branch Protection, Endpunktliste gegen Testliste	Julian Ammann
QZ-08	Übertragbarkeit	Das Gesamtsystem startet aus leerem Zustand mit <code>docker compose up</code> anhand der Betriebsdokumentation. Die Kernfunktionen sind in aktuellen Versionen von Chrome, Firefox und Safari sowie auf Smartphone-Displaybreite nutzbar.	Deployment-Test auf sauberer Umgebung durch nicht beteiligte Person, Playwright mit Chromium, Firefox und WebKit sowie manueller Safari-Prüfung	Kevin Jordan Taghu, Yin Yin Wu-Hanke
QZ-09	Zeitkorrektheit	Technische Zeitpunkte werden in UTC gespeichert; Geburtstage, Anlasstermine und Verschenkdaten bleiben reine Kalenderdaten. Benachrichtigungen berücksichtigen Europe/Berlin; Zeitumstellungen verschieben keinen Anlass auf einen anderen Kalendertag.	Unit- und Integrationstests mit fester Test-Uhr, Jahreswechseln, Schaltjahren und Zeitumstellungen	Kevin Jordan Taghu

3.2 Qualität des Softwaresystems

Die Qualitätsziele QZ-01 bis QZ-09 werden über die in der Konfiguration der Softwareentwicklung beschriebenen Maßnahmen erreicht: Teststrategie, lokale Git-Hooks, Continuous Integration, Code-Review und Definition of Done. Jede Änderung durchläuft dieselben Prüfschritte, bevor sie auf `main` gelangt.

Zeitpunkt	Prüfung	Nachweis
bei jedem Commit	Formatierung, Linting und Secret-Scan über die Git-Hooks	Ausgabe der Hooks
bei jedem Pull Request	Lint, Typprüfung, Unit- und Integrationstests, Build, Schwachstellenscan und mindestens eine Review-Freigabe; nach Aktivierung der Richtlinie blockieren Befunde nach den Regeln unter „Abhängigkeiten“	grüne CI und Freigabe im Pull Request
je Sprint	funktionale Abnahme der umgesetzten Anforderungen (QZ-01) und, sobald in der CI eingerichtet, Messung der Testabdeckung (QZ-07)	Rückverfolgbarkeitsmatrix, Abdeckungsbericht

Zeitpunkt	Prüfung	Nachweis
vor MS 4	End-to-End-Abläufe über mehrere Browser (QZ-08), Last- und Antwortzeitmessung (QZ-02), Barrierefreiheit (QZ-04), Sicherheitsprüfung (QZ-06) und Zeitkorrektheit (QZ-09)	Testabschlussbericht

Neben den automatischen Prüfungen werden Artefakte statisch geprüft. Die Konkretisierung der Anforderungen prüft das Team vor der Umsetzung (siehe Anforderungsmanagement). Architekturentscheidungen werden vor der Umsetzung als GitHub Issue mit Begründung und betrachteten Alternativen festgehalten und von einer zweiten Person freigegeben; Änderungen an Container-, Workflow- und Deploy-Konfiguration prüft Julian Ammann. Testfälle werden im selben Pull Request wie der Code reviewt. Die prüfende Person kontrolliert dabei, ob sie die Akzeptanzkriterien der genannten Anforderung abdecken und neben dem Normalfall auch Validierung und Fremdzugriff prüfen.

Zentrale Nachweise sind die Rückverfolgbarkeitsmatrix Anforderung ↔ Test ↔ Ergebnis (QZ-01), die in der CI gemessene Zeilenabdeckung von mindestens 70 % im Backend (QZ-07) sowie die grüne Continuous Integration mit bestandenem Review als Voraussetzung für jede Zusammenführung. Die abschließenden Messwerte stammen ausschließlich aus echten Läufen des Release-Stands und fließen in den Testabschlussbericht.

Wird ein Qualitätsziel verfehlt, wird die betroffene Änderung nicht zusammengeführt. Die Ursache wird behoben oder der Umfang wird angepasst, bevor weitergearbeitet wird.

3.3 Qualität der Liefergegenstände

Neben dem Softwaresystem werden auch die übrigen Liefergegenstände geplant geprüft, darunter Dokumente, Videos, Programmcode, das lauffähige System, der Testabschlussbericht und die Testkonten. Grundlage sind die Rollentabelle, die Prüfverfahren für Liefergegenstände und die Definition of Done aus der Konfiguration.

Für jeden Liefergegenstand gilt:

- Es sind eine verantwortliche und eine prüfende Person festgelegt. Die erstellende Person gibt ihren eigenen Liefergegenstand nicht allein frei.
- Die Prüfung erfolgt vor der Abgabe anhand des festgelegten Prüfverfahrens. Das Ergebnis wird im Pull Request oder am zugehörigen Redmine-Ticket festgehalten.
- Jeder Meilenstein hat einen eindeutig gekennzeichneten Abgabestand, auf den sich die Prüfung bezieht.

Für Dokumente und Präsentationen gelten zusätzliche Prüfpunkte:

Merkmal	Prüfung
Vollständigkeit	Alle für den Meilenstein geforderten Inhalte sind enthalten.

Merkmal	Prüfung
Fachliche Richtigkeit	Eine nicht beteiligte Person liest den Text gegen den aktuellen Stand von Code und Datenmodell.
Form	Einheitliche Struktur, Rechtschreibung, Format und Titelblatt werden geprüft.
Nachvollziehbarkeit	Aussagen sind mit Anforderungen, Artefakten oder dem Repository verknüpft.

Der Projektbericht (MS 6) wird zusätzlich über Turnitin eingereicht. Jedes Mitglied verantwortet seinen namentlich zugeordneten Textteil. Generierte Dokumente werden immer aus dem geprüften Release-Stand erzeugt, damit Dokumentation und System zusammenpassen.